NORTEX Code

Benutzerhandbuch

Version 0.1.0

Inhaltsverzeichnis

1 Willkommen bei Nortex Code 4
Was ist Nortex Code? 4
Wofür wird es verwendet? 4
Überblick der wichtigsten Funktionen 4
Zusammenhang mit der Sprache 4
2 Installation und erster Start 5
Systemvoraussetzungen 5
Installation 5
Starten 5
Erster Start 5
Dateien, die die IDE schreibt 5
Typische Probleme 5
3 Benutzeroberfläche 6
Menüleiste 6
Activity Bar 6
Explorer 7
Editorbereich 7
Tabs 7
Console (untere Panels) 7
Sidecar 7
Statusleiste 7
Weitere Ansichten 7
4 Projekte und Dateien 8
Projekt anlegen 8
Projekt öffnen / aktualisieren 8
Dateien 8
Nicht vorhanden 8
5 Arbeiten mit dem Editor 9
Grundfunktionen 9
Autovervollständigung 9
Zeilenbefehle 9
Suchen und Ersetzen 9
Tabs und geteilter Editor 10
Navigation 10
Umbenennen, Breadcrumbs und Minimap 10
6 Kompilieren und Ausführen 11
Auslösen 11
Ablauf 11
Ergebnis lesen 11
Overflow-Checks 11
Grenzen 12
7 Terminal und Konsolen 13
Terminal 13
8 Fehleranalyse und Debugging 14

Fehler erkennen 14
Diagnosen lesen 14
Typische Probleme 14
Debugging-Funktionen 14
9 Einstellungen und Anpassungen 16
Aufbau der Einstellungsansicht 16
Zurücksetzen, Export und Import 16
Aktive Einstellungen 16
Geplant (sichtbar, aber ausgegraut und ohne Wirkung) 17
Tastenkürzel 17
Konto und Synchronisierung 17
Fensteraufteilung 17
Nicht anpassbar 18
10 Tastenkombinationen 19
Allgemeine Bedienung 19
Dateiverwaltung 19
Editor 19
Navigation und Seitenleiste 20
Terminal und Console 20
Build, Ausführung und Debugging 20
Panels und Fenster 20
Menü-Kürzel 20
11 Praktischer Einstieg 21
12 Effizientes Arbeiten 22
13 FAQ und Fehlerbehebung 23

1 Willkommen bei Nortex Code

Inhaltsverzeichnis · Weiter: Installation

Was ist Nortex Code?

Nortex Code ist die grafische Entwicklungsumgebung (IDE, „Integrated Development Environment“) für die Programmiersprache Nortex. Eine IDE vereint Texteditor, Dateiübersicht, Compiler-Aufruf und Ausgabe in einem Fenster.

Nortex Code ist ein eigenständiges Windows-Programm. Zum Bauen ruft es den Nortex-Compiler nortexc auf, der .nx-Dateien in eine Windows-Programmdatei (EXE) übersetzt.

Wofür wird es verwendet?

- Nortex-Quelltext (.nx) schreiben,
- per Knopfdruck bauen und starten (Run, Strg+F5),
- Compiler-Meldungen mit Position lesen,
- mehrere Dateien in Tabs und geteilten Ansichten bearbeiten.

Überblick der wichtigsten Funktionen

Bereich	Funktion
Editor	Syntaxhervorhebung, Zeilennummern, Autovervollständigung, Suchen/Ersetzen, Split-Ansicht
Build	Run-Knopf, Overflow-Checks, nortex.toml-Projekte
Ausgabe	Compiler Log, Runtime Output, Terminal (PowerShell)
Anpassung	Theme, Dichte, UI- und Editor-Zoom

Nicht enthalten sind u. a. ein Debugger, Projekt-Assistenten und Datei-Umbenennen/Löschen (siehe Funktionsstand).

Zusammenhang mit der Sprache

Die Sprache bestimmt, was kompiliert werden kann. Heute baut der Compiler nativ nur ein kleines Teilset (u. a. int/bool, if/while/for, Funktionen und UTF-8-String-Slices für Variablen, Funktionsaufrufe und Console.writeLine). Die IDE erkennt mehr Syntax, als der Build tatsächlich erzeugen kann; Details: Sprachreferenz.

2 Installation und erster Start

Zurück · Inhaltsverzeichnis · Weiter: Oberfläche

Systemvoraussetzungen

- Windows 10/11, 64 Bit.
- Zum Linken erzeugter Programme: Microsoft-Linker (Visual-Studio-Build-Tools) – siehe Sprachreferenz, Kapitel 2.
- Optional: Git (für die Git-Anzeige) und PowerShell (für das Terminal).

Installation

Entpacke den Programmordner an einen beliebigen Ort. Neben nortex-code.exe müssen der Compiler nortexc.exe und die Bibliothek nortex_runtime.lib liegen. Fehlt etwas, meldet Build & Run das im Compiler Log; das Dashboard zeigt den Zustand der Werkzeuge.

Starten

nortex-code.exe doppelklicken. Der Arbeitsordner beim Start wird zum Projektordner (im Explorer-Feld angezeigt). Wer die EXE per Verknüpfung startet, setzt dort den gewünschten „Ausführen in“-Ordner.

Erster Start

Beim ersten Start öffnet sich das Dashboard (Projektübersicht mit Schaltflächen Neu, Datei öffnen, Build & Run und „Toolchain Diagnostics“). Dort steht, ob Compiler und Laufzeitbibliothek gefunden wurden. Beides sollte grün sein.

[Abbildung: Dashboard]

Dateien, die die IDE schreibt

- Deine Einstellungen (Theme, Dichte, Zoom, zuletzt geöffnete Dateien) im Ordner %APPDATA%\NORTEX Code.
- Beim Bauen: .exe neben der Quelldatei (ohne Manifest) bzw. der Ausgabeort laut nortex.toml.

Typische Probleme

Problem	Lösung
Compiler nicht gefunden	nortexc.exe neben nortex-code.exe legen
Laufzeitbibliothek nicht gefunden	nortex_runtime.lib neben die EXE legen
Linker-Fehler beim Build	Visual-Studio-Build-Tools installieren
Terminal startet nicht	Meldung „Shell nicht startbar“; PowerShell muss verfügbar sein

3 Benutzeroberfläche

Zurück · Inhaltsverzeichnis · Weiter: Projekte und Dateien

[Abbildung: Hauptfenster mit Editor]

Nr.	Bereich	Ort
1	Menüleiste	oben
2	Activity Bar	ganz links (schmale Icon-Leiste)
3	Seitenleiste (Explorer, Suche, Git, Debugger)	links neben der Activity Bar
4	Editorbereich mit Tabs und Run-Leiste	Mitte
5	Sidecar	rechts
6	Statusleiste	zwischen Editor und Konsole
7	Console (Compiler Log, Runtime Output, Terminal)	unten

Die Panels tragen nummerierte Titel (01 Explorer, 02 Editor, 03 Console, 04 Sidecar, 05 Einstellungen).

Menüleiste

Menüs: Datei, Bearbeiten, Ansicht, KI, Hilfe.

Menü	Einträge
Datei	Neue Datei (Strg+N) · Öffnen… · Ordner öffnen… · Speichern (Strg+S) · Speichern unter… · Projektordner schließen (blendet nur die Seitenleiste aus) · Projekt aktualisieren · Projektordner im Explorer öffnen · Beenden
Bearbeiten	Rückgängig · Wiederholen (ausgegraut, wenn nichts zu tun ist) · Ausschneiden (Strg+X) · Kopieren (Strg+C) · Einfügen (ausgegraut, nur per Strg+V im Editor) · Alles auswählen (Strg+A) · Suchen (Strg+F) · Suchen/Ersetzen (Strg+H) · Zeile duplizieren · Zeile nach oben/unten · Nächstes Vorkommen auswählen · Block-Kommentar (Strg+/) · Zu Zeile springen…
Ansicht	Zen-Modus (F11) · Explorer (Strg+B) · Terminal / Diagnosen (Strg+J) · Statusleiste anzeigen · Seitenleiste (Explorer, Suche, Git, Debugger) · Layout zurücksetzen · Zeilennummern anzeigen · Minimap anzeigen (Strg+Alt+M) · Zoom zurücksetzen · Sidecar (Strg+Umschalt+L) · Editor-Layout (Einzeln, Zwei Spalten, Zwei Zeilen, 2x2 Raster, Pane schließen) · Theme · Dichte · UI-Zoom · Editor-Zoom
KI	Hinweis „Noch nicht angebunden – es wird kein Code gesendet.“; alle Einträge (Code erklären, Refactoring-Vorschlag, Fehler analysieren, Tests vorschlagen, Dokumentation generieren) sind deaktiviert, geplant
Hilfe	Erste Schritte · Diagnoseinformationen kopieren · Lizenz · NORTEX Code-Handbuch · Dokumentationsordner öffnen · Tastatur-Kürzel · Update-Check (meldet nur, dass die Prüfung offline nicht verfügbar ist) · Info

[Abbildung: Menü Bearbeiten]

Activity Bar

Oben das N-Logo (Dashboard), darunter Explorer, Editor, Suche, Git, Debugger, ganz unten das Zahnrad (Einstellungen). Das aktive Symbol trägt einen 2 px breiten Streifen am linken Rand (violett für Explorer/Editor/Dashboard, blau für Git/Debugger). Ein Klick auf das bereits aktive Seitensymbol schaltet die Seitenleiste um (Strg+B).

Explorer

Zeigt den Projektordner als Baum mit einklappbaren Ordnern. Farbe des Symbols nach Dateityp (.nx violett). Klick öffnet die Datei. Oben: Eingabefeld für den Projektordner und Öffnen.

Editorbereich

Obere Zeile: Run (Strg+F5), Speichern, Kontrollkästchen Overflow-Checks, Dateiname. Danach folgen Tabs und Editor. Datei -> Speichern unter… öffnet den Dateidialog. Details: Kapitel 5.

Tabs

Ein Tab pro geöffneter Datei; + legt ein neues Dokument an. Der aktive Tab hat einen helleren Hintergrund und eine Amber-Linie. Ungespeicherte Änderungen zeigt ein Amber-Punkt • neben dem stets sichtbaren x (Schließen). Bei ungespeicherten Änderungen fragt ein Dialog, ob gespeichert oder verworfen werden soll. Ein roter Punkt ? markiert den aktiven Tab bei Compilerfehlern. Tabs lassen sich am Dateinamen ziehen, um sie neu zu ordnen oder in ein anderes Pane zu verschieben.

Console (untere Panels)

Reiter Compiler Log (n), Runtime Output, Terminal. Ein/Aus: Strg+J oder Ansicht -> Terminal / Diagnosen. Siehe Kapitel 7.

Sidecar

Rechtes Panel „KI-Assistenz“ mit Eingabefeld und Senden. Es ist an keinen KI-Anbieter angebunden. Anzeige: Strg+Umschalt+L oder Ansicht -> Sidecar. In schmalen Fenstern wird es automatisch eingeklappt.

Statusleiste

Von links nach rechts: Projektname · Statusmeldung (grün nach erfolgreichem Lauf, rot bei Fehlern) · Fehleranzahl · Zeile und Spalte. Sie lässt sich unter Ansicht -> Statusleiste ausblenden.

Weitere Ansichten

- Dashboard: Projektübersicht, Build- und Toolchain-Status, zuletzt geöffnete Dateien.
- Einstellungen: Zahnrad (Kapitel 9).
- Befehlspalette: Strg+Umschalt+P oder F1 (Kapitel 10).

[Abbildung: Befehlspalette]

Zen-Modus (Ansicht-Menü, F11) blendet Nebenbereiche aus; Esc beendet ihn. Kontextmenüs (Rechtsklick) gibt es nicht. Kleine Fenster gibt es für Zu Zeile springen, Hilfe -> Lizenz, Tastatur-Kürzel, Info, das Handbuch und das Konto-Fenster; für Datei öffnen/speichern dient der Windows-Dateidialog.

4 Projekte und Dateien

Zurück · Inhaltsverzeichnis · Weiter: Editor

Ein Projekt ist ein Ordner. Es gibt keinen „Neues Projekt“-Assistenten.

Projekt anlegen

1. Ordner im Windows-Explorer anlegen.
2. Optional eine nortex.toml darin ablegen (Manifest; siehe Manifest-Schema).
3. Nortex Code mit diesem Ordner als Arbeitsordner starten, oder den Pfad in das Feld oben im Explorer eintragen und Öffnen klicken.

Projekt öffnen / aktualisieren

Pfad im Explorer-Feld eingeben -> Öffnen. Meldung „Projekt geladen“ oder „Projekt konnte nicht aktualisiert werden: …“ (Statusleiste). Datei -> Projekt aktualisieren liest die Dateiliste im Hintergrund neu (auch nach Änderungen außerhalb der IDE). Mit einer nortex.toml im Projektordner zeigt die Statusleiste „Name -> Ausgabedatei“ und der Fenstertitel lautet „ — NORTEX Code“.

Dateien

Aktion | Vorgehen

Neu | + im Tab-Bereich, Datei -> Neue Datei oder Neu im Dashboard

Öffnen | Klick im Explorer · Strg+P (Windows-Dateidialog, Filter .nx/.nortex) · Datei öffnen im Dashboard · Datei auf das Fenster ziehen · zuletzt geöffnete Datei im Dashboard

Speichern | Strg+S oder Speichern. Bei einem neuen Dokument öffnet sich der Speichern-Dialog.

Speichern unter | Datei -> Speichern unter… oder Strg+Umschalt+S öffnet den Dateidialog.

Schließen | x im Tab. Bei ungespeicherten Änderungen kann gespeichert, verworfen oder abgebrochen werden.

Gespeichert wird atomar (ein fehlgeschlagener Schreibvorgang lässt die Originaldatei unverändert). Fehler erscheinen als Statusmeldung, z. B. „Speichern fehlgeschlagen: …“.

Nicht vorhanden

Umbenennen, Löschen, Ordner anlegen und Verschieben von Dateien gehen nur außerhalb der IDE (Windows-Explorer, Terminal-Tab). Danach Projekt aktualisieren. Der Explorer zeigt alle Dateien (außer .git) als Baum.

5 Arbeiten mit dem Editor

Zurück · Inhaltsverzeichnis · Weiter: Kompilieren und Ausführen

Grundfunktionen

- Syntaxhervorhebung für Schlüsselwörter, Bezeichner, Zahlen, Strings, Zeichen, Operatoren, Satzzeichen und Kommentare (Farben abhängig vom Theme).
- Zeilennummern in der linken Spalte.
- Rückgängig/Wiederholen: Strg+Z, Strg+Y oder Strg+Umschalt+Z (auch im Menü Bearbeiten).
- Fehler werden direkt im Editor markiert (Inline-Diagnosen, kurz nach dem Tippen aktualisiert) und im Compiler Log gelistet (Kapitel 8).
- Klammervervollständigung und Mehrfachcursor gibt es nicht.

Autovervollständigung

Ab 2 Zeichen eines Bezeichners öffnet sich ein Popup mit bis zu 5 Vorschlägen: erst Schlüsselwörter, dann Bezeichner aus dem aktuellen Dokument.

Taste | Wirkung

Pfeil hoch/runter | Auswahl bewegen (umlaufend)

Tab oder Enter | Vorschlag übernehmen

Esc | Popup schließen

Zeilenbefehle

Befehl | Taste

Kommentar ein/aus | Strg+/

Zeile(n) nach oben/unten | Alt+^ / Alt+v

Zeile(n) duplizieren | Umschalt+Alt+v

Nächstes Vorkommen des markierten Textes auswählen | Strg+D

Suchen und Ersetzen

Strg+F öffnet ein schwebendes Feld oben rechts im Editor, Strg+H zusätzlich mit „Ersetzen durch“. Ist Text markiert, wird er übernommen.

Bedienelement | Funktion

Suchfeld | Treffer werden live gesucht

Schalter Aa / W / .* | Groß-/Kleinschreibung · Ganzes Wort · Reguläre Ausdrücke (mit Capture-Gruppen im Ersetzungstext)

? / ? oder Enter / Umschalt+Enter | vorheriger / nächster Treffer

Ersetzen / Alle | aktuellen / alle Treffer ersetzen

x oder Esc | schließen

Ein ungültiger regulärer Ausdruck wird als Fehler angezeigt, ohne dass die IDE abstürzt. Dateiübergreifende Suche: Seitenleiste Suche (Strg+Umschalt+F): Suchbegriff eingeben, Suchen oder Enter; Klick auf einen Treffer öffnet die Datei (einfache Textsuche, keine Zeilenmarkierung).

Tabs und geteilter Editor

Mehrere Dateien liegen in Tabs; Strg+Tab wechselt zum nächsten Tab. Über Ansicht -> Editor-Layout (oder Tastatur) lässt sich der Editor teilen:

Layout | Taste

Zwei Spalten | Strg+\ (auch Strg+Umschalt+2)

Zwei Zeilen | Strg+Umschalt+\ (auch Strg+Umschalt+3)

2x2 Raster | Strg+Umschalt+4

Einzeln | Ansicht -> Editor-Layout -> Einzeln

Fokus: Strg+1 … Strg+4 wählt das Pane; Strg+Alt+Pfeil oder Alt+<-/-> wechselt in Richtung; Strg+Umschalt+W schließt das aktive Pane.

Navigation

Gehe„Gehe zu Definition“- und keine „Gehe zu Zeile“-Funktion. Verwenden: Strg+F, Projektsuche, Tabs, Strg+P (Datei öffnen).

Umbenennen, Breadcrumbs und Minimap

- F2 öffnet am Cursor ein Eingabefeld mit dem Bezeichner unter dem Cursor. Enter benennt alle Vorkommen in der aktuellen Datei um, Esc bricht ab. Kommentare, Strings und Schlüsselwörter bleiben unberührt; nur ASCII-Namen, kein projektweites Umbenennen.
- Breadcrumbs über dem Editor zeigen Datei und umschließende Scopes (z. B. struct und Funktion). Ein Klick springt zur Startzeile des Scopes.
- Minimap am rechten Rand (60 px); Klick scrollt. Ein/Aus mit Strg+Alt+M, über das Menü Ansicht oder die Befehlspalette; gespeichert als editor.minimap.enabled.

6 Kompilieren und Ausführen

Zurück · Inhaltsverzeichnis · Weiter: Terminal und Konsolen

Nortex Code hat eine Build-Funktion: Build & Run. Sie speichert, kompiliert mit nortexc build und startet die erzeugte EXE.

Auslösen

Weg	Hinweis
Knopf ? Run	im Editorbereich
Strg+F5	
Strg+Umschalt+B	gleiche Funktion (kein reiner Build)
Befehlspalette -> Build & Run	
Dashboard -> Build & Run	

Während des Bauens zeigt die Leiste „Kompiliere…“; ein zweiter Build kann gleichzeitig nicht starten.

Ablauf

1. Die aktive Datei wird gespeichert. Bei neuen Dokumenten ohne Pfad erscheint der Speichern-Dialog.
2. Ohne nortex.toml: nortexc build --output --overflow-checks=; die EXE liegt neben der Quelldatei.
3. Mit nortex.toml im Projektordner: gebaut wird das Manifest. Die EXE liegt im Unterordner build\ des Projekts; Overflow-Checks kommen aus der IDE-Einstellung, sofern das Manifest sie nicht festlegt.
4. Das Compiler-Log zeigt die Compilerausgabe. Danach startet die IDE die EXE; ihre Ausgabe steht im Runtime Output, gefolgt von „Programm beendet (Exitcode …)“.

[Abbildung: Ergebnis eines Laufs]

Ergebnis lesen

- Statusleiste grün: Build erfolgreich. Rot: Compilerfehler. Der Exitcode des Programms ist nur ein Wert (Nortex-Programme geben ihn über return aus) und färbt nie rot. Im Screenshot steht „Exitcode Some(0)“: das ist die tatsächliche Anzeige der IDE für Exitcode 0.
- Bei Fehlern: Reiter Compiler Log (Kapitel 8).

Overflow-Checks

Das Kontrollkästchen steuert --overflow-checks. An: Ganzzahl-Überlauf führt zu einem Trap (das Programm bricht ab). Aus: Überlauf läuft über (Wraparound). Standard: an in Debug-, aus in Release-Builds der IDE. Dieselbe Option steht in den Einstellungen und im Dashboard.

Grenzen

- Kein Abbrechen laufender Programme (Shift+F5 zeigt nur einen Hinweis).
- Kein Eingabekanal für laufende Programme.
- Nur der nativ unterstützte Sprachumfang baut (Sprachreferenz); alles andere endet mit NX2001/NX2002.

7 Terminal und Konsolen

Zurück · Inhaltsverzeichnis · Weiter: Fehleranalyse und Debugging

Das untere Panel 03 / CONSOLE hat drei Reiter:

Reiter	Inhalt	Direkt öffnen
Compiler Log (n)	Compilerausgabe und Diagnosen; n = Anzahl	Strg+Umschalt+M
Runtime Output	Ausgabe des gestarteten Programms und Exitcode	Strg+Umschalt+U
Terminal	echte PowerShell-Sitzung	Reiter anklicken

Das Panel ein-/ausblenden: Strg+J oder Ansicht -> Terminal / Diagnosen. Ein neuer Build leert Compiler Log und Runtime Output. Befehlspalette -> Clear Console leert beide ebenfalls.

Terminal

[Abbildung: Terminal]

- Beim ersten Öffnen des Reiters startet die Shell (powershell.exe) im Hintergrund; die Zeile zeigt „(aktiv)“ bzw. „(beendet)“ mit grünem/rotem Punkt.
- Befehl in das untere Eingabefeld tippen, Enter sendet. Die Ausgabe erscheint oben.
- Clear oder Strg+L löschen die Anzeige.
- Es ist keine vollwertige Konsole: keine interaktiven Programme, keine Farbcodes.
- Nur ein Terminal; mehrere Konsolen oder Terminal-Tabs gibt es nicht.
- Startet die Shell nicht, steht „Shell nicht startbar: …“.

8 Fehleranalyse und Debugging

Zurück · Inhaltsverzeichnis · Weiter: Einstellungen

Fehler erkennen

- Der Editor prüft den Text kurz nach dem Tippen und markiert Fehler direkt im Code.
- Der Reiter Compiler Log (n) listet alle Diagnosen; die Statusleiste zeigt „n Fehler“.
- Ein roter Punkt ? im aktiven Tab bedeutet: Datei hat Fehler.
- Nach einem Build rot/grün in der Statusmeldung.

Diagnosen lesen

Format im Compiler Log: Code [Datei:Zeile:Spalte] Meldung, darunter ggf. Hilfe: …. Rot = Fehler, Amber = Warnung.

Beispiel (Tippfehler summee, Zeile 3, Spalte 12):

```
NX1004 [main.nx:3:12] Unbekannter Bezeichner 'summee'. Der Wert wurde weder deklariert noc
h importiert.
  Hilfe: Prüfe die Schreibweise oder deklariere den Wert vor seiner Verwendung.
```

Behebung: Name korrigieren (summe). Ein fehlendes Semikolon meldet NX0100 („Nach einer Variablendeklaration wird ';' erwartet.“); die Position zeigt auf das nächste Token, das Semikolon fehlt also meist in der Zeile davor.

Code-Bedeutungen: Diagnosecodes und Sprachreferenz, Kapitel 11.

Typische Probleme

Meldung | Ursache / Lösung

NX2001 / NX2002 | Konstrukt wird geprüft, aber nativ noch nicht erzeugt (z. B. float, string-Variablen, Strukturen). Auf unterstützten Umfang beschränken

Linker-Fehler LNK2019 | Es fehlt public int main()

„Compiler … konnte nicht gestartet werden“ | nortexc.exe nicht gefunden (Kapitel 2)

„Build fehlgeschlagen (Exitcode …)“ | Fehler stehen im Compiler Log darüber

Debugging-Funktionen

Echtes Debuggen (Anhalten, Schrittbetrieb, Variablen inspizieren) ist nicht verfügbar. Der Grund: nortexc erzeugt keine Debug-Informationen (z. B. Zeilentabellen). Ohne sie kann die IDE ein laufendes Programm nicht an einer Quellzeile anhalten. Die IDE meldet das offen, statt eine Sitzung vorzutäuschen.

Vorhanden ist die Verwaltung von Haltepunkten und die Oberfläche dafür:

Funktion | Stand

Haltepunkt setzen/entfernen | F9 in der Cursorzeile. Die Datei muss gespeichert sein (neue Dokumente ohne Pfad: Hinweis in der Statusleiste). Die Statusleiste meldet „Breakpoint in Zeile n gesetzt/entfernt“.

Markierung im Editor | Ein ? in der Spalte links neben der Zeilennummer markiert Zeilen mit Haltepunkt.

Haltepunkt-Liste | Debugger-Panel (Activity Bar -> Debugger oder Strg+Umschalt+D): „Breakpoints (n)“

mit Datei:Zeile. Klick springt zur Zeile, x entfernt einen Eintrag, Alle entfernen leert die Liste.

Aufrufliste / Variablen | Das Panel enthält die Bereiche Aufrufliste und Variablen. Sie zeigen „Keine aktive Sitzung.“, weil noch kein Debugger angebunden ist.

F5 / ? Debuggen | Öffnet das Debugger-Panel und meldet, dass schrittweises Debugging nicht verfügbar ist. Die Sitzung bleibt inaktiv.

? Stopp | Beendet den (nicht vorhandenen) Sitzungszustand.

Umschalt+F5 | Meldet „Abbrechen laufender Programme ist nicht implementiert.“

Haltepunkte werden nur im laufenden Programm der IDE gehalten und nicht gespeichert; nach einem Neustart sind sie weg. Sie haben keinen Einfluss auf Strg+F5 (Build & Run).

Fehlersuche funktioniert derzeit über Diagnosen, Programmausgabe und den Exitcode (z. B. return eines Zwischenergebnisses, siehe Kapitel 11).

9 Einstellungen und Anpassungen

Zurück · Inhaltsverzeichnis · Weiter: Tastenkombinationen

[Abbildung: Einstellungen]

Öffnen: Zahnrad unten in der Activity Bar (nochmals klicken kehrt zum Editor zurück). Die Darstellungs-Regler stehen auch im Menü Ansicht.

Aufbau der Einstellungsansicht

- Suche (oben): sucht in Titel, Beschreibung und Stichwörtern. Alle eingegebenen Wörter müssen passen. Die Suche filtert auch die Tabelle der Tastenkürzel.
- Ebenenfilter: Alle Ebenen, Benutzer, Projekt oder Erweitert. Benutzer sind persönliche Vorgaben. Projekt ist für projektspezifische Werte gedacht; diese Einträge sind vorbereitet, aber noch ohne Wirkung (siehe „Geplant“). Erweitert enthält Spezialoptionen.
- 7 Kategorien: Darstellung · Editor · Tastenkürzel · Sprache und Code-Hilfe · Build und Debug · Konto und Synchronisierung · System, Sicherheit und Updates.
- Geänderte Einträge tragen einen Punkt •; ? setzt den einzelnen Eintrag auf den Standard zurück.

Zurücksetzen, Export und Import

Aktion | Wirkung

? am Eintrag | setzt diesen Eintrag zurück

Kategorie zurücksetzen | setzt alle aktiven Einstellungen der Kategorie zurück

Alle zurücksetzen | setzt alle aktiven Einstellungen zurück; die Liste der zuletzt geöffneten Dateien bleibt erhalten

Exportieren… | speichert die Einstellungen als JSON (Vorschlag nortex-code-settings.json)

Importieren… | liest eine JSON-Datei; ungültige Dateien werden mit einer Meldung abgelehnt, ungültige Einzelwerte fallen auf den Standard zurück. Die eigene Liste der zuletzt geöffneten Dateien bleibt unverändert

Die Einstellungen werden automatisch im Ordner %APPDATA%\NORTEX Code gespeichert (der Pfad steht am Ende der Ansicht). Beschädigte Dateien werden beim Start repariert.

Aktive Einstellungen

Einstellung | Kategorie | Standard | Wirkung

Farbschema | Darstellung | Industrial Midnight | Industrial Midnight, Professional Dark oder High Contrast. Befehlspalette -> Toggle Theme wechselt zum nächsten

Dichte | Darstellung | Comfortable | Compact, Comfortable oder Spacious: Abstände der Oberfläche

Oberflächen-Zoom | Darstellung | 100 % | 70–200 %; Strg+Mausrad ändert ihn in 10-%-Schritten

Editor-Zoom | Darstellung | 100 % | 70–200 %, Editorschrift

Statusleiste anzeigen | Darstellung | an | blendet die Statusleiste ein/aus

Zeilennummern anzeigen | Editor | an | Zeilennummernspalte

Minimap anzeigen | Editor | aus | auch Strg+Alt+M

Automatisch speichern | Editor | aus | speichert geänderte Dokumente mit Dateipfad höchstens alle 30 Sekunden; neue Dokumente ohne Pfad werden nicht gespeichert

Overflow-Checks | Build und Debug | an in Debug-Builds der IDE, aus in Release-Builds | Trap bei Ganzzahl-Überlauf; ein nortex.toml kann den Wert überschreiben. Gleiche Option wie im Editorbereich (Kapitel 6)

Der Zoom-Regler arbeitet in 10-%-Schritten und wirkt nach dem Loslassen; genauere Werte gibst du direkt ein. Eingegebene Prozentwerte werden erst mit Enter oder nach Verlassen des Feldes übernommen. Ungültige Werte in der Datei fallen beim Laden auf den Standard zurück.

Geplant (sichtbar, aber ausgegraut und ohne Wirkung)

Diese Einträge zeigt die Ansicht mit dem Hinweis „geplant“; der Mauszeiger nennt den Grund.

- Editor: Tabulatorbreite, Zeilenumbruch, Leerzeichen anzeigen, Beim Speichern formatieren, Klammerpaare hervorheben, Editorschrift
- Sprache und Code-Hilfe: Live-Diagnose, Automatische Vervollständigung (beide sind immer aktiv und noch nicht abschaltbar), Inline-Hinweise
- Build und Debug: Standard-Buildprofil, Zusätzliche Compiler-Argumente, Arbeitsverzeichnis (Projektebene fehlt), Prozess-Zeitlimit, Beim Start anhalten (Debugger noch nicht verfügbar, siehe Kapitel 8)
- Konto: Einstellungen synchronisieren
- System: Update-Kanal, Nutzungsdaten, Arbeitsbereich-Vertrauen. Zu den Nutzungsdaten: Es werden derzeit keine Daten erhoben.

Tastenkürzel

Die Kategorie zeigt die aktuelle Belegung als Tabelle. Anpassen ist noch nicht möglich. Vollständige Liste: Kapitel 10.

Konto und Synchronisierung

- Lokaler Modus: Es gibt kein Konto-System im Sinne eines Online-Dienstes. Die IDE läuft vollständig lokal; Editor, Compiler, Build und Projekte brauchen kein Konto.
- Der Menüpunkt Konto und der Knopf Nortex-Konto öffnen… zeigen den Status „Lokaler Modus“. Im Browser anmelden meldet, dass der Kontodienst noch nicht verfügbar ist. Es wird keine Anmeldung vorgetäuscht.
- Synchronisierung: Vier Schalter (Einstellungen, Themes, Tastenkürzel, Liste der Erweiterungen) sind nur vorbereitet. Sie werden gespeichert, wirken aber erst nach einer Anmeldung, die es noch nicht gibt. Standardmäßig sind alle aus.
- Quellcode bleibt lokal: Quellcode, Projekte und Dateien verlassen den Rechner nie; sie gehören nicht zu den synchronisierbaren Daten.
- Geplant (nur als ausgegraute Einträge sichtbar): KI-Funktionen, Erweiterungen, Lizenz, Geräte. Eine spätere Anmeldung erfolgt über den Systembrowser, ohne Passwörter in der IDE.

Fensteraufteilung

- Seitenleiste: Strg+B · Console: Strg+J · Sidecar: Strg+Umschalt+L · Zen-Modus: F11 oder Ansicht.
- Editor-Layout: Kapitel 5.

- Die Panelbreiten und -höhen sind in Grenzen per Ziehen anpassbar. Panels lassen sich nicht frei verschieben oder andocken.

Nicht anpassbar

Tastenkombinationen, Schriftart, Farben einzelner Elemente und Panelposition (Seitenleiste immer links, Sidecar rechts, Console unten).

10 Tastenkombinationen

Zurück · Inhaltsverzeichnis · Weiter: Praktischer Einstieg

Die Tasten gelten, solange die Befehlspalette geschlossen ist. Die Belegung lässt sich nicht ändern. Mit * markierte Kürzel erscheinen nicht in der Hilfe-Übersicht.

Allgemeine Bedienung

Taste | Funktion

Strg+Umschalt+P, F1 | Befehlspalette

Strg+Z | Rückgängig

Strg+Y, Strg+Umschalt+Z* | Wiederholen

Strg+Umschalt+X* | Erweiterungen – zeigt nur „Erweiterungen: nicht implementiert.“

F11 | Zen-Modus ein/aus

Esc | Zen-Modus verlassen (wenn aktiv)

In der Palette: tippen filtert, Pfeiltasten wählen, Enter führt aus, Esc schließt. Verfügbare Befehle: Build & Run, Toggle Theme, Clear Console, Toggle Zen Mode, Open File, Save File, Split Editor Columns / Rows / 2x2 Grid, Editor Single Layout, Close Editor Pane, Toggle Minimap.

Dateiverwaltung

Taste | Funktion

Strg+N | Neue Datei

Strg+S | Speichern

Strg+Umschalt+S | Speichern unter (Dateidialog)

Strg+P | „Schnellöffnen“: öffnet den Windows-Dateidialog (keine Dateiliste mit Suche)

Strg+Tab* | nächster Tab

Editor

Taste | Funktion

Strg+F | Suchen

Strg+H | Ersetzen

Strg+/ | Zeilenkommentar ein/aus

Strg+D | nächstes Vorkommen auswählen

Alt+^ / Alt+v | Zeile verschieben

Umschalt+Alt+v | Zeile duplizieren

F2 | Symbol umbenennen (aktuelle Datei)

Strg+Alt+M | Minimap ein/aus

Strg+Alt+B | main-Gerüst einfügen

^ v, Tab/Enter, Esc | im Vorschlags-Popup: wählen, übernehmen, schließen

Enter, Umschalt+Enter, Esc | im Suchfeld: nächster, vorheriger Treffer, schließen

Navigation und Seitenleiste

Taste | Funktion

Strg+Umschalt+E* | Explorer

Strg+Umschalt+F* | Suche

Strg+Umschalt+G* | Git

Strg+Umschalt+D* | Debugger-Panel

Strg+1 … Strg+4* | Editor-Pane 1–4

Strg+Alt+<-/->/^/v, Alt+<-/-> | Fokus auf Nachbar-Pane

Terminal und Console

Taste | Funktion

Strg+L | Terminalanzeige löschen (wenn der Terminal-Reiter aktiv ist)

Enter | Befehl senden

Strg+Umschalt+M* | Compiler Log (Probleme) zeigen

Strg+Umschalt+U* | Runtime Output zeigen

Build, Ausführung und Debugging

Taste | Funktion

Strg+F5 | Build & Run

Strg+Umschalt+B | Build & Run (identisch; kein reiner Build)

F5 | „Debugging starten“: öffnet das Debugger-Panel und meldet, dass schrittweises Debugging nicht verfügbar ist

Umschalt+F5* | Meldung „Abbrechen laufender Programme ist nicht implementiert.“

F9 | Haltepunkt in der Cursorzeile setzen/entfernen (Kapitel 8)

Panels und Fenster

Taste | Funktion

Strg+B | Seitenleiste ein/aus

Strg+J | Console ein/aus

Strg+Umschalt+L* | Sidecar ein/aus

Strg+\ , Strg+Umschalt+2* | zwei Spalten

Strg+Umschalt+* , Strg+Umschalt+3* | zwei Zeilen

Strg+Umschalt+4* | 2x2-Raster

Strg+Umschalt+W* | aktives Pane schließen

Strg+Mausrad | UI-Zoom

Menü-Kürzel

Die Menüs zeigen Kürzel nur dort, wo sie im Menütext stehen; die Einträge sind in Kapitel 3 aufgelistet. Zusätzlich gelten Strg+X, Strg+C, Strg+V und Strg+A direkt im Editor. Hilfe -> Tastatur-Kürzel öffnet ein Fenster mit der Kürzelübersicht, Hilfe -> NORTEX Code-Handbuch das Handbuch.

11 Praktischer Einstieg

Zurück · Inhaltsverzeichnis · Weiter: Effizientes Arbeiten

Ziel: ein Programm schreiben, bauen, einen Fehler beheben und speichern.

1. Starten: nortex-code.exe doppelklicken. Es erscheint das Dashboard.
2. Projektordner wählen: Activity Bar -> Explorer, Ordnerpfad in das Pfadfeld eingeben (z. B. C:\Nortex\erstes), Öffnen. Der Ordner muss existieren; ein „Neues Projekt“ kann die IDE nicht anlegen.
3. Neue Datei: Activity Bar -> Editor, im Tab-Streifen + klicken. Mit Strg+S oder Datei -> Speichern unter… den gewünschten Pfad und Dateinamen wählen (Endung .nx).
4. Programm schreiben:

```
public int main() {
    int summe = 2 + 3;
    return summe;
}
```

5. Speichern: Strg+S. Statusleiste: „Datei gespeichert“.
6. Bauen und ausführen: Strg+F5 (oder ? Run). Im Runtime Output steht „Programm beendet (Exitcode Some(5))“. Der Rückgabewert von main ist der Exitcode.
7. Fehler provozieren: return summe; zu return summee; ändern. Der Editor markiert die Stelle; Strg+F5 ergibt im Compiler Log NX1004 … Unbekannter Bezeichner 'summee', Statusleiste rot.
8. Beheben: zurück zu summe, Strg+F5 -> wieder Exitcode 5, Statusleiste grün.
9. Ausgabe ergänzen (optional; ein Stringwert je Aufruf):

```
using System;

public int main() {
    Console.writeLine("Hallo, NORTEX!");
    return 0;
}
```

Runtime Output: Hallo, NORTEX!.

10. Speichern: Strg+S. Der Punkt • im Tab verschwindet, sobald alles gespeichert ist.

Weitere lauffähige Beispiele: examples\language\ (Sprachreferenz).

12 Effizientes Arbeiten

Zurück · Inhaltsverzeichnis · Weiter: FAQ

- Kreislauf: schreiben -> Strg+F5 -> Compiler Log prüfen -> korrigieren. Die IDE speichert vor jedem Build selbst.
- Befehlspalette (Strg+Umschalt+P) statt Menüsuche.
- Fensteraufteilung: Editor mit Strg+\ teilen (Quelltext neben Beispiel/Test). Console mit Strg+J ein-/ausblenden, Strg+Umschalt+M/U springt zum passenden Reiter. Zen-Modus blendet Nebenelemente aus.
- Projekte: einen Ordner je Projekt, optional nortex.toml (Manifest) für Projektname, Einstiegsdatei und Build-Optionen; Fenstertitel und Statusleiste zeigen dann den Projektnamen.
- Navigation: Strg+P (Datei öffnen), Projektsuche Strg+Umschalt+F, Strg+Tab zwischen Tabs, Strg+F im Dokument.
- Zeilen bearbeiten mit Alt+^/v, Strg+/ und Strg+D statt Kopieren/Einfügen.
- Übersicht: kleine Funktionen, aussagekräftige Namen; die Autovervollständigung nutzt Bezeichner des Dokuments.
- Immer speichern, bevor die IDE geschlossen wird; es gibt keine Wiederherstellung ungespeicherter Tabs.

13 FAQ und Fehlerbehebung

Zurück · Inhaltsverzeichnis

Warum lässt sich ein Projekt nicht öffnen? Pfad prüfen (Ordner muss existieren). Drag & Drop nimmt nur .nx an („Nur .nx-Dateien werden geöffnet“). Ein defektes nortex.toml erzeugt eine Meldung in der Statusleiste.

Warum funktioniert die Kompilierung nicht?

1. „Compiler … konnte nicht gestartet werden“: nortexc.exe muss neben nortex-code.exe liegen oder im PATH sein.
2. Linker-/Runtime-Fehler: nortex_runtime.lib fehlt (neben der EXE, im Elternordner oder über NORTEX_RUNTIME_LIB).
3. Neue, noch nicht gespeicherte Dateien: erst mit Strg+S speichern und im Dateidialog einen Namen wählen.
4. NX-Fehler: siehe Kapitel 8.

Wo finde ich Fehlermeldungen? Im Tab-Reiter Compiler Log (Strg+Umschalt+M), im Editor markiert, Anzahl in der Statusleiste.

Wie öffne ich das Terminal? Reiter Terminal im unteren Panel; Panel mit Strg+J einblenden.

Wie speichere ich Dateien? Strg+S oder Datei -> Speichern; Strg+Umschalt+S für „Speichern unter“. Ungespeicherte Tabs zeigen •.

Wie ändere ich die Darstellung? Zahnrad -> Theme, Dichte, Zoom (Kapitel 9).

Mehrere Dateien gleichzeitig bearbeiten? Tabs und geteilter Editor (Kapitel 5).

Warum zeigt der Run-Button „Exitcode Some(0)“? So gibt die IDE den Exitcode 0 aus; kein Fehler.

Warum meldet die IDE bei float oder struct einen Fehler? Diese Konstrukte sind noch nicht nativ erzeugbar (NX2001/NX2002).

Gibt es Debugger, Git-Commit, Umbenennen/Löschen, KI? Nein, siehe Funktionsstand im Inhaltsverzeichnis.

Die Einstellungen sind zurückgesetzt. %APPDATA%\NORTEX Code\settings.json löschen; sie wird neu angelegt.